The mrm model after Chapter 5

A reading guide: where things are, and what the code computes

For Kenny Guo. 21 October 2026. Code references updated on 23 October 2026, after the readability pass of the model (`~/mrm-model`). The engine results are bit-identical to commit `3db791e`. Old names: `docs/RENAMES.md`.

How to use this guide

You know textbook chapters 0 to 5. This guide starts from that knowledge. It explains how the model works and shows where each part is in the code.

1. Section 1 gives five ideas that you need before you read any code. Read it first.
2. Section 2 defines the words that the code and this guide use.
3. Section 3 shows the whole model on one page.
4. Section 4 lists the core code in reading order.
5. Section 5 explains each physics part, from your Ch05 ring to the full model.
6. Section 6 lists what remains to learn and the first commands to run.

Equation numbers. "(eq. 5.26)" means equation 5.26 in textbook chapter 5. This guide always writes the equation out. "Not in the textbook" marks an equation that the textbook does not have.

Numbers. Every model number comes from the preset `g26`: the file `mrm/params/presets/g26.yaml`, or the command `mrm params explain g26 <key>`. The textbook's own numbers are not verified, so do not use them as design values. A row "Book" tells you where the textbook uses another value or another method. "My arithmetic" marks a number that I calculated from model values. No model output gives it directly.

Sign convention. The model detuning is $d = \lambda_{res} - \lambda_{line}$. Ch05 uses $\delta = \lambda_L - \lambda_r$, so $d = -\delta$. A positive d means that the resonance is red of (at a longer wavelength than) its laser line. The laser is then on the blue flank of the dip. This is the stable lock side. Two sign conventions are in use, so always say where the resonance is relative to the laser. The code writes i for your j. Time dependence is $e^{+i\omega t}$, and one round trip multiplies a field by $a\,e^{-i\phi}$. This is your Ch05 convention.

1 Five ideas you need first

The numbers here are for ring 0 of g26, cold and at its own resonance. The model gives these inputs:

- loaded $Q_L = 3850$ (the Q of the whole ring, with every loss, `ring.optics.q_loaded`), FSR $= 1.8\,\mathrm{THz}$ (`ring.optics.fsr_thz`), resonance wavelength $\lambda = 1311.04\,\mathrm{nm}$, so $f = c/\lambda = 228.67\,\mathrm{THz}$;
- power coupling per pass $\kappa_1^2 = 0.09402$ (through) and $\kappa_2^2 = 0.00145$ (drop), round-trip amplitude $a = 0.94784$;
- so $t_1 = \sqrt{1 - 0.09402} = 0.9518$, $t_2 = \sqrt{1 - 0.00145} = 0.99927$, and $t_1 t_2 a = 0.9518 \times 0.99927 \times 0.94784 = 0.9015$.

The product $t_1 t_2 a$ is the fraction of the field that one lap keeps. For the add-drop ring it takes the place of the ta of the all-pass ring.

1.1 The comb of dips: FSR, linewidth, finesse and Q

Plot the through power against wavelength (Figure 1). You get a comb of narrow dips. Four numbers describe the comb.

1. **Where the dips are.** A dip occurs when a whole number m of wavelengths fits around the ring: $\phi = 2\pi m$ (eq. 5.9).
2. **FSR** (free spectral range): the spacing between two dips. $\Delta f_{FSR} = c/(n_g L)$ (eq. 5.14) and $\Delta\lambda_{FSR} = \lambda^2/(n_g L)$ (eq. 5.13). Model: $\Delta\lambda_{FSR} = \lambda^2 \Delta f_{FSR}/c = (1311.04\,\mathrm{nm})^2 \times 1.8\,\mathrm{THz}/c = 10.32\,\mathrm{nm}$.
3. **Linewidth** (FWHM): the width of one dip at half depth. $\Delta\lambda_{FWHM} = (1-ta)\lambda^2/(\pi n_g L\sqrt{ta})$ (eq. 5.21). In frequency this is $\Delta f_{FWHM} = (1-ta)\Delta f_{FSR}/(\pi\sqrt{ta}) = 0.0985 \times 1800\,\mathrm{GHz}/(\pi \times 0.9495) = 59.4\,\mathrm{GHz}$, which is 0.341 nm. The model's exact value is 59.39 GHz, 340.5 pm (`fwhm_pm`).
4. **Finesse** $\mathcal{F}$: the FSR divided by the linewidth, $\mathcal{F} = \pi\sqrt{ta}/(1-ta)$ (eq. 5.25). It says how sharp the dips are compared with their spacing. It depends only on ta, the field kept per lap. Model: $\mathcal{F} =$

1800 GHz/59.39 GHz = 30.3.

5. **Q** (quality factor): the light's own frequency divided by the linewidth, $Q = \lambda/\Delta\lambda_{FWHM} = f/\Delta f_{FWHM}$ (eq. 5.22). Model: $Q = 228.67\,\text{THz}/59.39\,\text{GHz} = 3850$.

How they are linked. Divide (eq. 5.22) by (eq. 5.25): $Q = \mathcal{F} \times f/\Delta f_{FSR}$. Model: $30.3 \times (228.67/1.8) = 30.3 \times 127.0 = 3850$. Equation 5.23 gives the same link with $\sqrt{ta}$ set to 1.

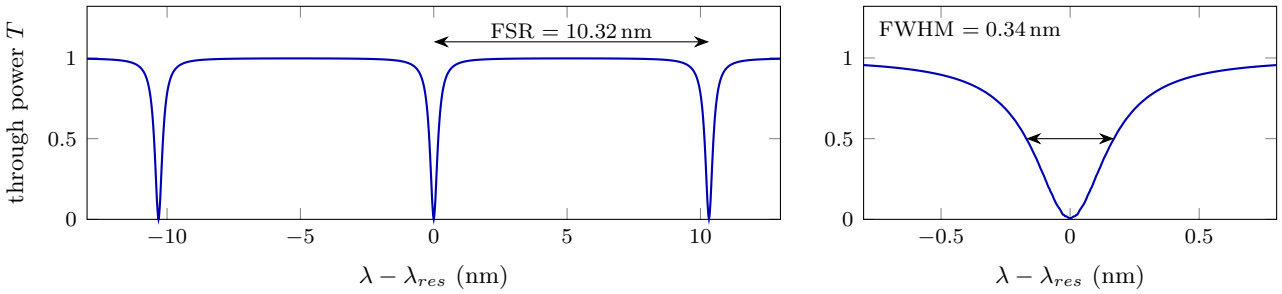


Figure 1: Through power of ring 0 from (eq. 5.30), with the model's t_1, t_2, a. Left: the comb. The dips repeat every FSR. Right: one dip, enlarged 16 times, with the FWHM arrow at half depth. The finesse is the ratio of the two arrows: $10.32/0.34 \approx 30$.

1.2 Photon lifetime and steady state

The RC picture. Apply a voltage step V_s to an RC circuit. The capacitor voltage is $v(t) = V_s\,(1 - e^{-t/RC})$. It settles to a constant value. After $4.6\,RC$ it is within 1 % of V_s, because $e^{-4.6} = 0.01$. "Steady state" means this settled condition: nothing changes any more.

The ring stores energy like a capacitor. Let U be the optical energy in the ring. Ch05 (section "Quality factor, finesse, lifetime, build-up", part "Finesse, photon lifetime, build-up") derives the free decay. Start from $Q = \omega U/P$ (eq. 5.22), with lost power $P = -dU/dt$:

$$\frac{dU}{dt} = -\frac{\omega U}{Q} \;\Rightarrow\; U(t) = U(0)\,e^{-t/\tau_p}, \qquad \tau_p = \frac{Q}{\omega} = \frac{Q\lambda}{2\pi c} \text{ (eq. 5.26).}$$

τ_p is the *photon lifetime*. Ch00 calls it the ring's ringdown time. Model: $\tau_p = 3850 \times 1.311 \times 10^{-6}\,\text{m}/(2\pi \times 2.998 \times 10^8\,\text{m/s}) = 2.68\,\text{ps}$.

Check from the laps. One lap takes $T_{rt} = 1/\Delta f_{FSR} = 1/1.8\,\text{THz} = 0.556\,\text{ps}$. Per lap the field keeps $t_1 t_2 a = 0.9015$, so the power keeps $0.9015^2 = 0.8128$. After n laps $U_n = U_0\,(0.8128)^n$. So $\tau_p = T_{rt}/(-\ln 0.8128) = 0.556\,\text{ps}/0.2073 = 2.68\,\text{ps}$. This agrees. In laps, the lifetime is $\tau_p/T_{rt} = \mathcal{F}/2\pi = 30.3/6.28 = 4.8$ laps.

The build-up after the input turns on. The field decays twice as slowly as the energy, with time constant $2\tau_p$ (Ch05 states this). The lap sum (eq. 5.6) shows the build-up. After n laps, the field misses the terms of lap n and later. Their sum is exactly $(t_1 t_2 a)^n$ times the final field. So the missing part shrinks by 0.9015 per lap. That is the RC step response, with $RC = 2\tau_p = 5.4\,\text{ps}$ (Figure 2). The field is within 1 % of its final value after $\ln 100/(-\ln 0.9015) = 44$ laps, which is $44 \times 0.556 = 24.7\,\text{ps}$.

Steady state of the ring. After that time the stored energy is constant, so the through and drop powers are constant. Equation 5.30 gives the powers in this state only:

$$T_{through} = \frac{t_2^2 a^2 - 2t_1 t_2 a\cos\phi + t_1^2}{1 - 2t_1 t_2 a\cos\phi + (t_1 t_2 a)^2}, \qquad T_{drop} = \frac{(1-t_1^2)(1-t_2^2)\,a}{1 - 2t_1 t_2 a\cos\phi + (t_1 t_2 a)^2} \quad \text{(eq. 5.30).}$$

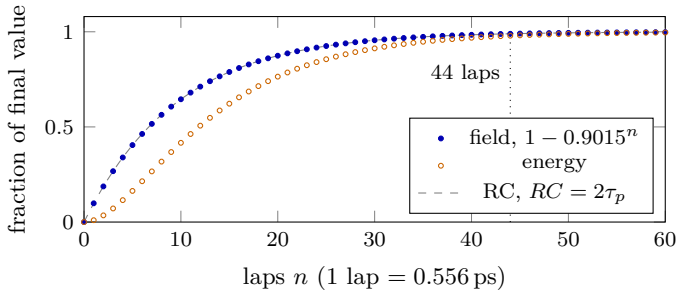


Figure 2: The ring fills like an RC circuit. The dots are the lap sum (eq. 5.6) cut after n laps. The dashed line is the RC step response with $RC = 2\tau_p = 5.36\,\text{ps}$, which is 9.6 laps. The two agree. After 44 laps (24.7 ps) the field is within 1 % of its final value. The energy is the square of the field.

1.3 Why the model can use the steady-state formulas

The model is a program that moves time forward in fixed steps. One step is a *tick*. One tick is 1 μs (`sim.dt_plant_s`).

- One tick holds $1\,\mu s / 0.556\,\text{ps} = 1.8$ million laps.
- The ring settles in about 44 laps (24.7 ps). So at every tick the ring is long settled.
- So the program does not add laps one at a time. It does not simulate the light as it travels. Once per ring and per tick, it puts the present ϕ into (eq. 5.30).

What changes between ticks is the temperature. The silicon index rises with temperature. In the model the effective index rises by 1.88×10^{-4} per kelvin at 25 °C (`ring.optics.dn_eff_dt_per_k`). So ϕ rises and the resonance moves red. Your shift formula $\Delta\lambda_r = (\lambda/n_g)\,\Delta n_{eff}$ (eq. 5.32) gives the slope. With the small thermal expansion of the ring added, the model has 65.1 pm/K at 25 °C and 73.9 pm/K at the lock temperature (134 °C). Ch09 covers this effect. Heat changes slowly, like the voltage of an RC circuit. The thermal time constants of the model go from 0.4 μs to several milliseconds.

So each tick does three things:

1. Update every temperature, like one RC step over 1 μs.
2. Compute ϕ from the new temperature.
3. Put ϕ into (eq. 5.30).

EE picture. The optics is a resistor divider: its output follows its input at once, and it has no memory. The heat is a capacitor: it keeps its value from one tick to the next.

1.4 Self-heating and the Newton solve

Self-heating. The ring loses part of its light. This is the loss a. By energy conservation the lost power is $P_{in}(1 - T_{through} - T_{drop})$, with both powers from (eq. 5.30). Most of it is absorbed and becomes heat:

$$P_{abs} = \eta\, P_{in}\,(1 - T_{through} - T_{drop}), \qquad \eta = 0.848 \text{ (the absorbed share of the loss).}$$

The heat raises the ring temperature, like a current through a resistor raises a voltage. Over one tick the model uses

$$T = T_{other} + g\, P_{abs}, \qquad g = \sum_j \left(1 - e^{-h/\tau_j}\right) r_j \quad \text{(not in the textbook).}$$

Here $h = 1\,\mu s$ is the tick. T_{other} is the temperature that the ring would have at the end of the tick with no absorbed light in this tick. Each pole j is one RC section of the heat path, with DC gain r_j and time constant τ_j. The factor $1 - e^{-h/\tau_j}$ is how far that RC section charges in one tick. Model (ring 0, light to ring): $g = 4.34(1 - e^{-1/0.4}) + 2.31(1 - e^{-1/3.18}) + 3.10(1 - e^{-1/13.0}) + 0.29(1 - e^{-1/3000}) = 3.98 + 0.62 + 0.23 + 0.00 = 4.83\,\text{K/mW}$. The DC value (the sum of all r_j) is 10.0 K/mW. This guide calls this whole step "the heat model over one tick".

Why a few kelvin matter. One linewidth is 340.5 pm/73.9 pm/K = 4.6 K. At the lock each ring absorbs about 1.5 mW (the model's lock solve `solve_locked_steady_state`, 25 °C, 1.05 V bias). At DC that gives $10.0\,\text{K/mW} \times 1.5\,\text{mW} = 15\,\text{K}$, about 1.1 nm of resonance shift.

The loop. Temperature sets ϕ. ϕ sets P_{abs}. P_{abs} sets the temperature. The optics has no delay inside a tick.

So the end-of-tick temperature must satisfy one equation per ring:

$$f(T) = T - T_{other} - g\,P_{abs}\big(\phi(T)\big) = 0.$$

The function has a cosine inside a fraction, so there is no algebraic solution. The program uses *Newton's method* (Figure 3):

1. Start from the previous tick's temperature T_0.
2. Draw the tangent line of f at T_0. Its zero crossing is the next guess: $T_1 = T_0 - f(T_0)/f'(T_0)$.
3. Repeat until $|f(T)| < 10^{-7}$ K (`DEFAULT_SH_TOL_K` in `mrm/plant/plant.py`). The code allows at most 10 Newton steps. With the evaluation at T_0, that is at most 11 evaluations.

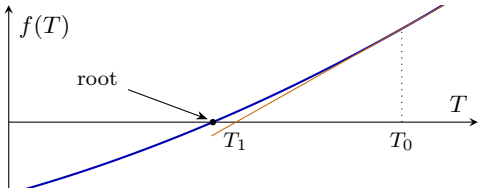


Figure 3: One Newton step (a sketch). The tangent at T_0 crosses zero at T_1, close to the root. Each step roughly squares the error near the root. The model stops at $|f| < 10^{-7}$ K. That is 10^{-7} K $\times$ 73.9 pm/K $= 7 \times 10^{-6}$ pm of resonance, far below the 340 pm linewidth.

How accurate and how fast. Near the root, each step roughly squares the error. In a steady g26 run I measured 1 to 2 Newton steps per ring and tick, mean 1.8 (my measurement, 3000 ticks).

Near a fold. With strong self-heating, one heater power can give two stable ring temperatures. Ch10 calls this *bistability*. Each stable solution is a *branch*. A branch ends at a *fold*, where $f'(T) \to 0$. There a plain Newton step can jump to the wrong root. So the code shortens each step to a distance that cannot contain a root. It always follows the root that the ring was already on (section 5.9).

1.5 What one tick does

First, four parts of the hardware:

- **Monitor photodiode (PD):** a germanium photodiode on each ring's drop port. Its current is 0.95 A/W $\times$ the drop power.
- **ADC** (analog-to-digital converter): turns the PD current into an integer, the *ADC code*. It has 16 bits (codes 0 to 65 535). One code is $40\,\mu A/65\,535 = 0.610$ nA.
- **Heater:** a resistor inside each ring. Its power heats the ring and moves the resonance red.
- **DAC** (digital-to-analog converter): turns an integer, the *heater code*, into a voltage. It has 16 bits. Code 0 gives 0 V and code 65 535 gives 1.6 V.

At each tick (1 μs), for every ring:

1. At every 10th tick (100 kHz), your controller runs. It reads the ADC codes that arrived since its last run. It writes new heater codes. A code reaches the DAC after a delay (latency) of 1 μs.
2. The scenario sets the outside inputs: ambient temperature, disturbance heat, laser powers and frequencies, data on or off.
3. The DAC turns the heater code into a voltage. The heater power is $P = V^2/R$.
4. The phase model computes ϕ at each laser line. It uses the ring temperature and the data voltage on the ring's PN junction (section 5.3).
5. The heat model over one tick moves every temperature forward. For each ring, in the order that the light meets them, the Newton solve of section 1.4 finds the end-of-tick temperature. The light that a ring passes goes on to the next ring.
6. The monitor PD collects the drop current. Every 4 ticks the ADC makes one code. The code reaches the controller 2 μs later.
7. The program records the true state, for example the true detuning. Your controller never sees it.

The loop closes like this: heater code $\to$ heater power $\to$ ring temperature $\to \phi \to$ drop power $\to$ ADC code $\to$ your controller $\to$ heater code.

2 Words you will meet

Word	Meaning
The program	
tick, controller edge	One time step of 1 μs. A controller edge is a tick at which your controller runs (every 10th tick).
kernel	The compiled inner loop that runs one tick (`mrm/sim/kernel.py`).
numba, `@njit`	A tool that compiles a Python function to machine code the first time it runs. `@njit` marks such a function.
NumPy twin	`kernel_ref.py`: the same tick in plain NumPy. It is slow but readable. A *parity test* checks that both give the same bits.
preset, leaf, key	A preset is a YAML file with every device number. A leaf is one number in it. Its key is its path, for example `ring.optics.q_loaded`.
provenance	A word that says where a number comes from. *printed*: in a Lightmatter publication. *read-off*: read from a published figure. *calibrated*: fitted to data by the model. *inferred*: worked out from published facts. *solver*: from a mode or heat simulation. *literature*: from other papers. *assumed*: a reasoned guess. *derived*: computed from other leaves.
relations R1–R5	The formulas in `mrm/params/relations.py` that compute derived numbers. R1: thermo-optic slope. R2: heater efficiency. R3: geometry and FSR. R4: Q values and loss. R5: latencies.
`realize`, `PlantParams`	`realize()` converts a preset to SI units for each world. It returns one read-only object, `PlantParams`.
world, nominal, corner	A world is one complete set of device numbers, shared by every ring of the die. The nominal world has every number at its preset value. A corner is a named world with some numbers at an extreme.
variation, LHS	Variation is the ring-to-ring spread inside one world. LHS (Latin hypercube sampling) spreads N random worlds evenly over every range.
Truth, oracle	Truth is the true state (for example the true detuning). The program knows it, your controller does not. The oracle is a benchmark that reads Truth and finds the best stable operating point. You may not use it as your controller.
warm start, cold start	A warm start begins with every ring near its lock. A cold start begins with the heaters off.
start protocol	How the first tick picks the self-heating branch: `branch_low`, `branch_high` or `laser_ramp`.
diagnostic	A check number that the model computes from the preset, for example `fwhm_pm`.
excursion	A time interval in which a ring is out of lock.
harness, telemetry	A harness is a simpler test setup outside the engine. Telemetry is the debug output of a controller.
test and tag names	A1–A4, B3, B5: Lightmatter's acceptance tests (Ch18). A1 is the aggressor test. CAL-n, VAL-n, BND-n: the model's calibration, validation and bound checks. "07 P0-1", "07 D9", "Q4": items in `docs/research/07_design_critique.md`.
knowledge tokens	Names in the preset that say which design numbers your controller may read.
latch, FIFO	A latch is a register that holds a command until the hardware takes it. A FIFO (first in, first out) is a queue. Here it is a delay line for a command.
trace, metrics	A trace is a recorded time signal. Metrics are the summary numbers of a run.
The device and the physics	
G25, G26, imec	G25 and G26 are generations of Lightmatter's device. imec is a research institute. The model borrows its thermal data.
WDM, line	Wavelength-division multiplexing: 8 laser lines share one waveguide, the *bus*. Each ring modulates its own line.
foreign line, wrong line	A foreign line is the line of another ring. A wrong-line ring has locked onto a foreign line.
bus order	The order in which the light meets the rings (ring 0 first).
NRZ	Non-return-to-zero data. Each bit holds one of two levels for one bit time.
PN junction	A diode across the ring waveguide. Reverse bias removes free carriers, so n_{eff} rises and the loss falls (Ch06).

Word	Meaning
OMA, relOMA	OMA (optical modulation amplitude) is the difference between the output powers for a "1" and a "0". relOMA is the OMA divided by the ring's input power.
extinction ratio	The ratio of the "1" and "0" output powers.
BER	Bit error ratio: the fraction of received bits that are wrong.
side-of-fringe lock	A lock with the laser on a flank of the dip, not at its bottom. There a small shift gives the largest change of power.
thermo-optic effect	The index of silicon rises with temperature (Ch09).
self-heating, branch, fold, bistability	See section 1.4 and Ch10.
self-heating relief	The factor by which self-heating divides the heater's effect on the red side (section 5.8).
heater authority	The range of heat that the heater can supply. It can only add heat, never remove it.
TCR	Temperature coefficient of resistance: the fractional change of the heater resistance per kelvin.
supply limit	All heaters share one supply. Above its power or current limit, every heater gets less power.
thermal pole	One first-order lag of a heat path, like one RC section: a DC gain r (K/W) and a time constant τ.
Foster sum	A heat path written as a sum of poles, $\sum_k r_k/(1 + s\tau_k)$: parallel RC sections in series.
ZOH, FOH	Zero-order hold: the input is constant across a tick. First-order hold: the input changes linearly across a tick.
explicit, implicit	An explicit update uses the previous tick's value. An implicit update solves for the end-of-tick value (section 1.4).
crosstalk	Heat from one heater or ring that reaches another ring.
aggressor	An on-die heater next to ring 0. The test drives it as a disturbance (scenario A1).
dark current	The current of a photodiode with no light. It grows with temperature.
TCMT	Temporal coupled-mode theory, an approximate ring model. g26 does not use it.

3 The model on one page

What it is. The model simulates the device that your lock controller will drive: the transmit bank of Lightmatter's G26 link. Eight microring modulators sit on one bus waveguide. Each ring has a heater, a PN junction for the data, and a monitor PD on its drop port. Eight laser lines sit on a 400 GHz grid near 1310 nm. The data is 50 Gb/s NRZ. The model advances in ticks of 1 μs. Your controller runs every 10 ticks (100 kHz). It reads ADC codes and writes heater DAC codes, as firmware does. Section 1.3 explains why each tick can use the steady-state ring formulas.

How to find things. Run these from `~/mrm-model`:

```
.venv/bin/mrm params explain g26 <key>    # one number: value, source, worlds, textbook value ("disagrees:")
                                          # <key>: a leaf (ring.thermal.tau_fast_s) or a diagnostic (fwhm_pm)
grep -n "^def \|^class " mrm/plant/optics.py            # function map of a module (read its docstring first)
grep -n "q_loaded" mrm/params/r*.py                     # where a leaf enters the code (grep the last part of its key)
grep -n "# ---" mrm/sim/kernel.py                       # sections of the compiled tick
grep -n "textbook\|disagrees" mrm/params/presets/g26.yaml   # where the model departs from the book
grep -rn "07 P0-1" mrm docs                             # follow a tag of docs/research/07_design_critique.md
```

Read these four documents first, in this order.

1. `docs/EXPLAINED.md`: a plain overview of the model, written for you.
2. `docs/KNOWN_ISSUES.md`: what is still wrong or out of date.
3. `docs/ENGINE.md`, sections 3–6: how one simulation step runs.
4. `docs/ARCHITECTURE.md`, section 2: which folder holds which part.

To see a function in use, open its test file. Most files in `mrm/plant/` have a test file `tests/verification/test_<name>.py`. For example, `optics.py` has `test_optics.py`.

To look up a number, use `mrm params explain` (above), or open `docs/PARAMETERS.md`.

Do not trust these yet. They are out of date. Each one is listed in `docs/KNOWN_ISSUES.md`.

- The preset `g26_calibrated`. It was fitted before the last fixes.

- The two validation reports, `docs/VALIDATION.md` and `docs/VALIDATION_g26_calibrated.md`.
- The pass of the 30-hour soak test (A4). It must run again.
- The comment at the top of `experiments/examples/a1_pi_lock.yaml`. It says 350 pm. The model now gives 454–473 pm.

Where things are.

`mrm/params/`	Presets (`presets/g26.yaml`), the loader, the relations R1–R5, and `realize.py` (makes the worlds and `PlantParams`).
`mrm/plant/`	The physics of one tick: `phase.py`, `optics.py`, `selfheat.py`, `thermal.py`, `electrical.py`, `sensing.py`, `laser.py`, `datapath.py`.
`mrm/scenarios/`	The disturbances (A1 and A2 aggressor, soak, laser drift, traffic, faults) and the aggressor calibration.
`mrm/control/`	The controller interface, the knowledge tokens, and `baselines/` (shipped controllers: `pi_lock`, dither, `wdm_supervisor`, …).
`mrm/sim/`	The compiled tick (`kernel.py`), its NumPy twin (`kernel_ref.py`), the runner, the metrics, the experiments, the command line.
`mrm/validation/`	Published targets, the oracle, the validation report (out of date).
`experiments/`, `examples/`	Experiment files (`examples/`, `acceptance/`). Controller templates and `tutorial.ipynb`.
`docs/`	`EXPLAINED.md`, `ENGINE.md`, `ARCHITECTURE.md`, `PARAMETERS.md`, `KNOWN_ISSUES.md`, `handoff/` (one design note per module), `research/`.
`runs/`	One folder per run: `report.txt`, `metrics.json`, `trace.h5`, `experiment.resolved.yaml`.
`tools/`, `webviz/`	`memguard.py` (stops a job above 3 GB), `clear_numba_cache.py`. A 3-D and 2-D viewer of the plant.

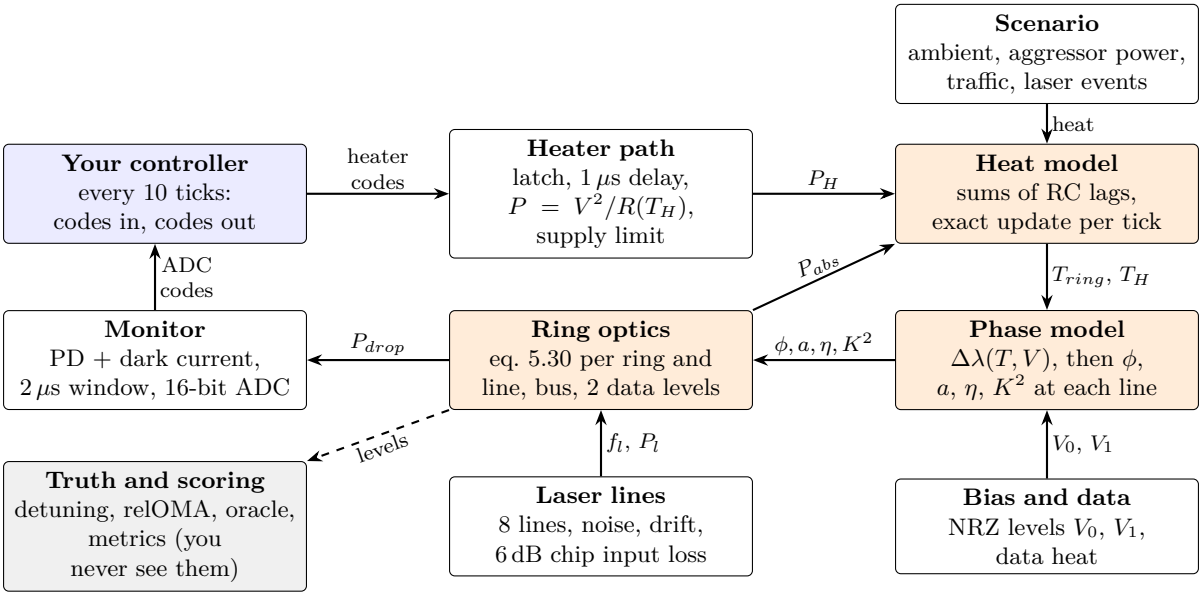


Figure 4: The signal flow of one tick. The three orange blocks are solved together inside each tick, ring by ring in bus order, with the Newton solve of section 1.4. The monitor also gets the PD temperature (not drawn). The code order in one tick is: controller edge, latch, scenario, laser, NRZ levels, phase model, heater, heat and self-heating step, monitor, Truth, metrics.

4 Core lines of code to read, in order

Line numbers are for the tree after the readability pass of 23 October 2026. Read the docstrings first. Skip `mrm/sim/kernel.py` until you need it. It calls the leaf modules, and it copies the self-heating step line for line. `kernel_ref.py` is the readable version of the same tick.

#	File and symbol	Lines	What to take from it, and why
A. Your side of the interface (start here, to know the goal)			
1	`examples/controllers/template_python_controller.py` `MyIntegralLock`	28–59	The file you copy to start. A plain integral loop holds the monitor code at a fraction of a known peak code. It is the smallest working controller.
2	`mrm/control/api.py` docstring; `ControllerSpec`, `Observation`	1–47; 157–536, 633–672	What your controller gets and returns. `reset(spec)` runs once. `step(obs)` runs at each controller edge and returns integer codes.
3	`mrm/core/channels.py` `V1_CHANNEL_TABLE`	403–468	The name, unit and rate of every signal, for example `tx.heater` and `tx.monitor.samples`. Look up a signal name here.
4	`mrm/control/baselines/pi_lock.py` docstring	1–110	The shipped lock. Which side of the dip to lock on and why, the start-up steps (MAP, RETURN, LOCK), the PI law, the failure modes. Read it before you design your own.
B. Optics: from your Ch05 ring to the model ring			
5	`mrm/plant/optics.py` docstring; `bus_propagate` docstring	22–84, 103–110; 3078–3094	The sign conventions and the four ports (0 in, 1 through, 2 drop to the monitor, 3 add). The bus, and the average over the two data levels.
6	`mrm/plant/optics.py` `_airy_line`	531–572	Your add-drop ring (eq. 5.30), exact: T, D, the loss fraction, and their slopes in ϕ. The core of the optics.
7	`mrm/plant/phase.py` docstring, equations 1–6	25–88	Where ϕ comes from: how temperature and voltage move the resonance, and how that gives ϕ, a, η, K^2 at each line.
8	`mrm/plant/phase.py` `thermal_shift_m`, `resonance_shift_ws`, `_fill_level`	236–240, 268–278, 289–320	The same equations as code. Use it to check one equation against its code.
9	`mrm/params/relations.py` docstring R3, R4	27–37, 55–69	How the preset's FSR, Q values and loss become L, m, K_t^2, K_d^2 and a. This is where t_1, t_2, a come from.
10	`mrm/plant/optics.py` `ring_lines_nb`	1040–1139	One ring at all 8 lines and both data levels. It averages over the data and adds up the absorbed and monitor powers.
11	`mrm/plant/optics.py` `ring_exact`	2855–2910	A Python function for one ring. Call it in a notebook to compare the line shape with Ch05.
C. Modulation			
12	`mrm/plant/electrical.py` `depletion_shift_m`, `analytic_junction_table_from_ring`	1574–1588, 1623–1651	How the junction voltage changes n_{eff} and the loss (a square-root law). This is how data moves the resonance.
13	`mrm/plant/electrical.py` `nrz_levels_row`	701–736	The two junction voltages V_0, V_1 from the bias and the swing.
14	`mrm/sim/kernel.py` `_truth_a` (relOMA lines)	1451–1457	relOMA $= \lvert P_{thru,1} - P_{thru,0} \rvert / P_{in}$ at the ring's own through port. The score uses it.
D. Heat: self-heating, heat paths, heater			
15	`mrm/plant/selfheat.py` docstring	1–85	The per-ring self-heating solve of section 1.4, how it stays on its branch, the start protocols.
16	`mrm/plant/selfheat.py` `solve_ring_temperature`	130–226	The Newton solve. It starts from the previous root and never jumps over a root.
17	`mrm/plant/selfheat.py` `lorentzian_folds`, `bistability_ratio`	530–559	Where the folds are, and the ratio that says whether a ring is bistable.
18	`mrm/plant/thermal.py` docstring; `foh_scalar`	1–78; 164–186	Heat paths as sums of RC lags, and the exact update of one lag over one tick.
19	`mrm/plant/thermal.py` `InterimRom` (docstring)	761–817	The heater-to-ring and light-to-ring paths, and the crosstalk rules. Its default numbers are not the g26 numbers.
20	`mrm/plant/thermal.py` `crosstalk_tau_s`, `site_fraction`	860–900	How crosstalk gets weaker and slower with distance.
21	`mrm/plant/electrical.py` docstring; `heater_power_row`; `supply_limit_row`	1–72; 423–531; 534–561	From heater code to heater power: the DAC, $P = V^2/R$ with R that changes with temperature, the shared supply limit.
E. Monitor			
22	`mrm/plant/sensing.py` docstring	1–89	From drop power to ADC code, with the noise of each sample. This is what one monitor code means.

#	File and symbol	Lines	What to take from it, and why
23	`mrm/plant/sensing.py` `monitor_tick_row`	337–467	Each tick the monitor collects current. Every 4 ticks it makes a code and queues it with its release tick.
24	`mrm/control/baselines/pi_lock.py` `monitor_block_mean`, `block_inputs`	312–333	How a controller averages the codes that arrived since its last step.
F. The engine (the program that runs the ticks)			
25	`mrm/core/timing.py` docstring	1–71	Time in integer ticks. Each delay has one owner. The order of events at a controller edge.
26	`mrm/sim/kernel_ref.py` `ReferenceEngine.step`	238–423	The readable NumPy tick. It gives the same bits as the kernel. Step through it in a debugger.
27	`mrm/sim/kernel.py` `_world_loop`	2013–2545	The compiled tick. Section markers: 2040 controller edge, 2214 optics inputs, 2316 heater, 2387 self-heating, 2451 sensors, 2509 Truth, 2522 metrics.
28	`mrm/sim/runner.py` `run`	159–281	One run: build the plant, warm start, reset the controller, run, collect metrics and traces.
G. Scoring, scenarios and parameters			
29	`mrm/validation/oracle.py` docstring	1–75	How the best stable operating point is found, and the OMA penalty against it. The engine's OMA-penalty metric does not import this file. It finds a similar point itself (`kernel.py` `_oracle_a`), with the OMA at the ring's own through port.
30	`mrm/sim/metrics.py` docstring; `summarize`	1–65; 740–818	Lock error, lock state, excursions, relOMA, wrong line. These are the columns of the report.
31	`mrm/sim/experiment.py` `_execute` (self-reference pre-run)	1501–1525	How the example files measure each ring's lock error from its own operating point.
32	`mrm/scenarios/library.py` `a1_slide10_aggressor`	133–188	The A1 aggressor: its temperature curve, its site, and how it is referenced.
33	`mrm/scenarios/calibrate.py` docstring	1–54	How the aggressor power is found that makes ring 0 follow the published curve.
34	`mrm/params/presets/g26.yaml` sections	234–1482	fidelity 234, laser 280, ring 339 (optics 418, thermal 650, heater 878, junction 972, monitor 1003), controller_interface 1144, metrics 1162, uncertainty 1169, variation 1448.
35	`mrm/params/realize.py` docstring; `realize`	1–40; 741–1174	How a preset becomes `PlantParams`: worlds, ring spread, SI units, relations, heat paths, latencies in ticks.
36	`experiments/examples/steady_pi_lock.yaml`, `a1_pi_lock.yaml`	whole files	An experiment file: preset, controller, scenario, start state, scoring. Copy one to make your own.

5 The bridge: from your ring to the whole model

Each part ends with a short table. "Code" gives the place in the code. "g26" gives the model numbers. "Book" gives the difference from the textbook.

5.1 The model ring is your add-drop ring

The model uses your add-drop equations (eq. 5.30) exactly, without the Lorentzian approximation. For each ring and each of the 8 lines, `_airy_line` computes the through power T, the drop power D and the loss fraction $\mathcal{L}$, per watt of input:

$$t_j = \sqrt{1-K_j^2}, \qquad k_j = \sqrt{K_j^2}, \qquad a = e^{-\alpha_i L/2}, \qquad \mathcal{D}(\phi) = 1 - 2t_1t_2a\cos\phi + (t_1t_2a)^2,$$

$$T(\phi) = \frac{t_1^2 - 2t_1t_2a\cos\phi + t_2^2a^2}{\mathcal{D}(\phi)}, \qquad D(\phi) = \frac{k_1^2k_2^2\,a}{\mathcal{D}(\phi)}, \qquad \mathcal{L}(\phi) = \frac{k_1^2}{\mathcal{D}(\phi)}\,(1-a)\,(1+t_2^2a), \qquad P_{abs} = \eta\,\mathcal{L}\,P_{in}.$$

T and D are (eq. 5.30). The cosine makes each formula periodic in ϕ, so one evaluation covers every resonance order. The loss term $\mathcal{L}$ is not in the textbook. It has three factors:

- $k_1^2/\mathcal{D}$ is the power that circulates just after the input coupler. At resonance it is your build-up factor $\kappa^2/(1-ta)^2$ (eq. 5.27), with ta replaced by t_1t_2a.
- $1-a$ is the power fraction lost in one half of the ring.

- $1 + t_2^2 a$ adds the second half: the power $t_2^2 a$ continues past the drop coupler.

Only the fraction η of that loss is absorption that heats the ring. The rest is scattering.

The preset does not give t_1, t_2 and a. It gives the loaded Q, the drop share κ_d/κ and a loss budget. `relations.py` (R4) converts them with the exact energy decay over one round trip T_{rt} (not in the textbook):

$$K_j^2 = 1 - \exp(-\omega T_{rt}/Q_j), \quad a = \exp\left(-\frac{\omega T_{rt}}{2Q_i}\right), \quad \frac{1}{Q_L} = \frac{1}{Q_i} + \frac{1}{Q_t} + \frac{1}{Q_d}, \quad Q_d = \frac{Q_L}{\kappa_d/\kappa}.$$

The sum of reciprocals is your $1/Q \approx 1/Q_{int} + 1/Q_c$ (eq. 5.24), with a third channel for the drop coupler.

Ch05	Model	Code name	Note
$\kappa_1^2,\ \kappa_2^2$	$K_t^2,\ K_d^2$	`k2_th`, `k2_dr`	Power coupling per pass, taken at the LINE wavelength.
a, ϕ	a, ϕ	`a_rt`, `phi`	ϕ is absolute: about $2\pi \times 79$ at resonance. Coupler phase $\psi = 0$.
$n_g L/c$	$T_{rt} = 1/\mathrm{FSR}$	`t_rt`	Group delay of one round trip.
Q, Q_{int}, Q_c	Q_L, Q_i, Q_t, Q_d	`q_loaded`, …	Loaded, intrinsic, through-coupler and drop-coupler Q.
$\delta = \lambda_L - \lambda_r$	$d = \lambda_{res} - \lambda_{line}$	`detuning_m`	$d = -\delta$. Half width $w = \Delta\lambda_{FWHM}/2$ (HWHM).
$d\phi/d\lambda$ (eq. 5.12)	$\partial\phi/\partial\lambda_{res}$	`dphi_dlamres`	Same physics, opposite sign: $\partial\phi/\partial\lambda_{res} = +2\pi n_g L/\lambda^2$. `kernel.py` line 353.
$\Delta\lambda_r$ (eq. 5.32)	$\Delta\lambda$	`resonance_shift_ws`	The resonance shift. `phase.py` line 269.

Hard point. The drop coupler is a weak tap: $Q_L/Q_d = 3850/550\,000 = 0.007$. So the through port acts almost like your all-pass ring. The drop port takes about 1 % of the input at the peak. Check it with $T_{drop,max} = \kappa_1^2\kappa_2^2 a/(1 - t_1 t_2 a)^2$ (eq. 5.31): $0.09402 \times 0.00145 \times 0.9478/0.0985^2 = 0.013$, so 1.3 %. The linewidth of (eq. 5.31) is 341 pm, almost the same as without the drop port (my arithmetic).

Code `optics.py` `_airy_line` 531–572, `ring_lines_nb` 1040–1139; `relations.py` 55–69. A TCMT form (`fidelity.optics_model: tcmt`, a sum over the 3 nearest orders) exists. g26 does not use it.

g26 $Q_L = 3850$, Q_i 7451–7543, Q_t 7978–8084, $Q_d = 5.5 \times 10^5$, K_t^2 0.0940–0.0963, K_d^2 0.00145–0.00147, $a = 0.94784$, $\eta = 0.848$, FWHM 336–341 pm, $L = 42.27\,\mu m$ ($R = 6.727\,\mu m$), $n_g = 3.94$, $T_{rt} = 0.5556$ ps. Under-coupled at the cold resonance: $t_1 = 0.952 > t_2 a = 0.947$. The G25 device shows a 27.9 dB notch at 0 V.

Book Q 3500, FWHM 374 pm, n_g 4.2, R 6.3 μm. The model uses 3850, the middle of 3500–4200.

5.2 Where the resonance is: the phase model

Your shift formula $\Delta\lambda_r = (\lambda/n_g)\Delta n_{eff}$ (eq. 5.32) moves a resonance. The model computes this shift once per ring and tick, from the ring temperature T and the junction voltage V (`phase.py` equation 1, not in the textbook):

$$\Delta\lambda(T, V) = s_0\Big[(T - T_{ref}) + \tfrac{\beta}{2}\big((T - T_0)^2 - (T_{ref} - T_0)^2\big)\Big] + \frac{\lambda_{res}}{n_g}\Big[f\,\Delta n_{eff}(V) - f\,\Delta n_{eff}(V_{ref})\Big].$$

The first term is the heat (section 5.7). The second term is the junction (section 5.3). The model then moves the whole cold phase curve by that shift. This is a "rigid comb": every order moves by the same amount. It reads the phase at each laser line (equation 2, not in the textbook):

$$x = \lambda_l - \Delta\lambda, \qquad \Delta = x - \lambda_{res}, \qquad n(x) = n_{eff} + (n_{eff} - n_g)\frac{\Delta}{\lambda_{res}} - \frac{n_g'\,\Delta^2}{2\lambda_{res}}, \qquad \phi = \frac{2\pi\, n(x)\, L}{x}.$$

- $\phi = 2\pi m$ (eq. 5.9) exactly when the shifted resonance reaches the line.
- Each ring is built 7.6 nm blue of its own line: $\lambda_{res}^{cold} = \lambda_{line} - \Delta\lambda_{off}$. The heater must close this gap.
- The order is $m = \mathrm{round}(n_{eff} L/\lambda_{res})$. R3 then sets $n_{eff} = m\lambda_{res}/L$, so the cold resonance sits exactly at the offset.
- The couplings are taken at the line: $K_j^2(\lambda_l) = K_{j,0}^2 \exp[\partial_\lambda \ln K_j^2\,(\lambda_l - \lambda_{res})]$. So under lock the dip depth does not change with the heater code.
- The loss follows the junction: $\alpha_i = [\alpha_{i,ref} + f\alpha_{fca}(V) - f\alpha_{fca}(V_{ref})]\, e^{c_R(T - T_{ref}) + c_H(T_H - T_{ref})}$, and $\eta = \alpha_{heat}/\alpha_i$ (equation 5). In g26 $c_R = c_H = 0$.

Hard point. The rigid comb keeps the cold FSR in wavelength when the ring heats. The locked order is exact. The adjacent orders are off by ±33 pm at the 7.6 nm lock (KNOWN_ISSUES item 15). This error reaches only the terms of the foreign lines.

Code `phase.py` docstring 25–88, `_fill_level` 289–320, `coupling_row` 364–378; `realize.py` line 1224 (cold resonance).

g26 Ring 0 on 1318.6 nm … ring 7 on 1302.6 nm. Cold resonances 1311.0 … 1295.0 nm. $m = 79$ (ring 7: 80). FSR 1.8 THz = 10.19–10.44 nm at the lines. $T_{ref} = 25\,°C$, $V_{ref} = 0$ V.

Book Keeps a measured FSR bound above 14.1 nm. The model treats that bound as unsupported and uses the 1.8 THz design value.

5.3 Modulation: the PN junction and the two data levels

Free carriers lower the silicon index and absorb light (the plasma-dispersion effect, Ch06). A PN junction runs around a fraction f of the ring. Reverse bias widens the depletion region (the zone with no free carriers). So n_{eff} rises, the resonance moves red, and the loss falls. In an abrupt junction the depletion width grows as $\sqrt{V + V_{bi}}$, with V_{bi} the built-in voltage. So the model uses a square-root law (not in the textbook). Here $m = 0.5$ is the depletion exponent, not the resonance order:

$$\Delta\lambda_J(V) = A\big[(V + V_{bi})^m - V_{bi}^m\big], \qquad A = \frac{\mathrm{eff}_0\, V_{bi}^{1-m}}{m}, \qquad \frac{d\lambda}{dV} = A\, m\, (V + V_{bi})^{m-1},$$

$$\Delta n_{eff}(V) = \frac{n_g}{\lambda}\,\Delta\lambda_{full}(V), \qquad \alpha_{fca}(V) = \alpha_0 - \frac{4\pi q}{\lambda}\,\Delta n_{eff}(V).$$

- eff_0 is the shift per volt at 0 V. α_0 is the free-carrier loss at 0 V.
- $q = \lambda\Delta\alpha/(4\pi\Delta n)$ is the scaled ratio of loss change to index change.
- $\Delta\lambda_{full} = \Delta\lambda_J/f$ is the shift of a junction around the full ring. The code multiplies the index and loss change by f again at run time.
- The data swings the junction between $V_0 = V_b - V_{pp}/2$ and $V_1 = V_b + V_{pp}/2$ (bias V_b, swing V_{pp}). Level 1 has the higher reverse voltage, so it is redder and has less loss.
- The ring is solved at both levels every tick. Every output is the average over the ones density p_1: $\bar{X} = p_1 X(\phi_1, a_1, \eta_1) + (1 - p_1)X(\phi_0, a_0, \eta_0)$.

Hard point. Bias moves the ring in two ways at once. The index change moves it red. The loss change alters the absorbed heat, and that moves it back. At the g26 lock a 0.2 V bias step moves the resonance only 0.1–0.5 pm. The index law alone gives about 10 pm. This is an open physics check. Data also heats each ring by about 2 K (junction charging and the driver). So traffic start, traffic stop and squelch (driver off) are thermal steps.

Code `electrical.py` `depletion_shift_m` 1574–1588, `analytic_junction_table_from_ring` 1623–1651, `nrz_levels_row` 701–736, `data_activity_heat_row` 936–970. `materials.py` `plasma_dispersion` 707–731 is used offline only.

g26 $A = 137.6\,\mathrm{pm/V^{1/2}}$, $V_{bi} = 0.9$ V, $f = 0.8$, $q = 0.06$. So $\mathrm{eff}_0 = Am\,V_{bi}^{m-1} = 68.8/0.949 = 72.5$ pm/V at 0 V, and $68.8/\sqrt{4.65} = 32$ pm/V at 3.75 V. Bias 1.05 V, swing 1.3 Vpp, so $V_0 = 0.40$ V and $V_1 = 1.70$ V. The two levels sit $137.6 \times (\sqrt{2.6} - \sqrt{1.3}) = 65$ pm apart by the index law (my arithmetic).

Book Interpolates the G25 bias ladder up to 1.6 V and lists a flat 50 pm/V. Lists 0.9 Vpp as the bench swing. The model reads 0.9 V as the pattern-generator swing and uses 1.3 V. No absorbed-heat term in the bias shift.

5.4 OMA and relOMA

The receiver decides between the two data levels. So the useful signal is the difference of the two through powers, the OMA (none of these equations is in the textbook):

$$\mathrm{relOMA} = \frac{|P_{thru,1} - P_{thru,0}|}{P_{in}}, \qquad \mathrm{OMA}_{bus} = |P_{out,1} - P_{out,0}|,$$

$$\mathrm{BER} = \tfrac{1}{2}\,\mathrm{erfc}\big(Q/\sqrt{2}\big), \quad Q = q_{ref}\,\mathrm{OMA}_{rx}/\mathrm{OMA}_{sens}.$$

Here Q is the receiver's signal-to-noise measure, not the ring's Q. q_{ref} is its value at the receiver sensitivity OMA_{sens}. For a small level separation, the OMA is about $P_{in}\,|dT/d\lambda|$ times the separation. So the OMA

follows the slope of the dip at the operating point, not the depth of the dip.

Hard point. A deep dip (large extinction ratio) does not give a large OMA. Sketch the dip and mark the two resonance positions on it. At the bottom of the dip the two powers are almost equal.

Code `kernel.py _truth_a` 1451–1457 (relOMA), 2446–2448 (bus OMA and BER); `datapath.py ber_from_oma` 204–212.

g26 About −6.1 dB at the lock (CAL-14: −6.07 dB), against the printed −2.1 dB. This target fails (KNOWN_ISSUES item 30). `docs/research/07_design_critique.md` D9 notes that OMA in dBm at the actual +4 dBm input would close the gap. That reading is the open question Q4.

Book relOMA = OMA / 1 mW at the modulator, which is that second reading. The model divides by P_{in} at the ring, on its own line (07 D9).

5.5 The operating point: a side-of-fringe lock

For a Lorentzian dip, the slope $|dT/d\lambda|$ is largest at a distance FWHM/$(2\sqrt{3})$ from the resonance, about 0.29 FWHM. The OMA is largest there. Near that point the OMA loss grows as the square of the lock error. The monitor also gives a signed error on one flank. So the lock point is a set-point on a flank, not the resonance. `pi_lock` sets its target at a fraction f of the measured drop peak. For a pure Lorentzian that puts the ring at (not in the textbook)

$$d_{set} \approx w\sqrt{1/f - 1} \qquad (w = \text{HWHM}).$$

The oracle (section 5.15) finds the exact optimum with the exact ring, the bus, both levels and self-heating.

Hard point. Both flanks give almost the same OMA. The best point on the blue side has 92–94 % of the best OMA. Only the red side ($d > 0$) is stable with self-heating (sections 5.8 and 5.9). Use FWHM/$(2\sqrt{3})$ as the first estimate and the oracle as the exact answer.

Code `oracle.py` docstring 21–38; `pi_lock.py` docstring "Which side, and why" 22–32.

g26 Oracle optimum $d = +103$ to $+105$ pm. FWHM/$(2\sqrt{3}) = 340.5/3.46 = 98$ pm (my arithmetic). `pi_lock` uses $f = 0.7$, so $d_{set} = w\sqrt{1/0.7 - 1} = 0.65w = 0.65 \times 170 = 111$ pm. The settled warm start sits at +97 to +121 pm. The 5 pm lock budget is $5/340 = 1.5$ % of a linewidth.

Book $\delta_{opt} = 374/(2\sqrt{3}) \approx 108$ pm, from its unverified Q.

5.6 WDM: eight rings on one bus

Eight lines share one bus, at $f_k = f_c + (k - \frac{L-1}{2})\Delta f$ and $\lambda_k = c/f_k$. Light meets ring 0 first. Each ring sees the averaged through power of the rings before it, times the loss of the bus segment. Lines add as powers, because different frequencies do not interfere (not in the textbook):

$$P_{in,k,l} = P_{chip,l}\, t^2_{seg,k} \prod_{j \text{ before } k} t^2_{seg,j}\, \bar{T}_{j,l}, \qquad P_{drop,k} = \sum_l \bar{D}_{k,l}\, P_{in,k,l}, \qquad P_{abs,k} = \sum_l \overline{\eta\mathcal{L}}_{k,l}\, P_{in,k,l}.$$

Every ring absorbs and drops all 8 lines. So each monitor sees a small background from the foreign lines. In the engine this bus walk runs inside the self-heating solve, ring by ring in bus order.

Hard point. The ring comb (one FSR, about 10.3 nm) and the line comb (2.29 nm) have different periods. A cold ring sits 7.6 nm blue of its own line, about 2 FWHM blue of line $n + 3$. A heater ramp from cold crosses foreign lines. Self-heating can capture the ring on one of them. A single-ring loop cannot see the difference. A supervisor (`wdm_supervisor`) assigns the lines at start-up. To see it, draw both combs and slide the ring comb red from 7.6 nm blue.

Code `realize.py line_frequencies_hz` 597–611; `phase.py bus_t2_from_plant_params` 215–231; `optics.py bus_propagate` docstring "The bus, and the single-pass bound" 3078–3094, `selfheat_commit` 1586–1616; `kernel.py` wrong-line block 1471–1497.

g26 Line 0 has the lowest frequency (1318.6 nm). Ring k is on line k. Segment transmission 0.99916, so ring 7 gets 0.59 % less light than ring 0. +4 dBm per line at ring 0 (10 dBm at the laser module, less 6 dB chip input loss). The next order sits 2.8–3.0 FWHM from a line. Above about 113 °C ambient a ring needs one more FSR of heating.

Book Ch07 sizes the heater for a generic 4.5 nm blue offset. It also quotes Lightmatter's measured 7.6 nm, which the model uses. 14 dBm per line at the laser, against 10 dBm. Both give +4 dBm at the ring.

5.7 The thermo-optic shift

Heat raises the silicon index, so the resonance moves red (Ch09). Your shift formula (eq. 5.32) with $\partial n_{eff}/\partial T$ and the thermal expansion α_L of the ring gives the slope $s = d\lambda_r/dT$. The slope grows with temperature. The ring locks at the temperature where the integral of s equals the built-in offset (not in the textbook):

$$s(T) = \frac{\lambda}{n_g}\Big(\frac{\partial n_{eff}}{\partial T} + n_{eff}\,\alpha_L\Big) = s_0\big[1+\beta(T-T_0)\big], \qquad \int_{T_{ref}}^{T_{lock}} s(T)\,dT = \Delta\lambda_{off} \ (\text{mod FSR}_\lambda).$$

Check at 25 °C (my arithmetic): $s_0 = (1311\,\text{nm}/3.94)\times(1.88\times10^{-4}+2.45\times3\times10^{-6}) = 332.7\,\text{nm}\times1.95\times10^{-4} = 65\,\text{pm/K}$. Each tick, the code expands the same law about the previous ring temperature (`local_thermo_optic`). The self-heating solve then uses the exact local slope.

Code `relations.py` R1 38–41; `phase.py` `thermal_shift_m` 236–240, `local_thermo_optic` 243–251; `optics.py` `_sh_prepare` 1542–1561.

g26 $s_0 = 65.075\,\text{pm/K}$ at 25 °C, $\beta = 1.2377\times10^{-3}/\text{K}$, so $s = 65.075\times(1+1.2377\times10^{-3}\times109.4) = 73.89\,\text{pm/K}$ at the lock. Lock at 134.38 °C (about 109 K of heating) at 25 °C ambient. One world uses one s law for every ring. World H86 (86 pm/K, constant): `--alt uncertainty.thermo_optic=H86`.

Book 86 pm/K held constant, and $dn_{Si}/dT = 1.86\times10^{-4}/\text{K}$ (a C-band value). The model uses its 1310 nm solver value.

5.8 Self-heating: an inner feedback loop

Section 1.4 gave the idea. Here is the loop view (Ch10). The absorbed power peaks near resonance, like the drop curve. Heat moves the resonance red, and that changes the absorbed power again. Treat this as an inner proportional loop around the ring temperature (none of these equations is in the textbook):

- Z_0 is the DC thermal resistance from the heater to its ring (K/mW).
- z says that one milliwatt of absorbed light heats the ring z times more than one milliwatt of heater power.
- The self-heating gain G_{sh} converts absorbed power to resonance shift.
- u is the detuning that the heater alone would give.

$$G_{sh} = s(T)\,Z_{opt}, \quad Z_{opt} = z\,Z_0; \qquad \text{stable if and only if } G_{sh}\frac{\partial P_{abs}}{\partial\lambda_{res}} < 1; \qquad \frac{dd}{du} = \frac{1}{1+G_{sh}\,|P'_{abs}|} \ \text{(red side)}.$$

On the red side ($d > 0$), heat moves the resonance away from the laser. So P_{abs} falls and opposes the heat. This is negative feedback. The *relief* $1+G_{sh}|P'_{abs}|$ divides the effect of the heater and of every other disturbance. On the blue side the feedback is positive. The code solves the loop implicitly, with the Newton solve of section 1.4:

$$F(T_k) = T_k - T_k^{other} - g_k\,P_{abs,k}(T_k) = 0, \qquad g_k = \sum_j (1-a_j)\,r_j^{opt}, \quad a_j = e^{-h/\tau_j}.$$

An explicit update (with the previous tick's P_{abs}) oscillates with a period of 2 ticks at 1 μs steps. So the code does not use one. A start from the previous root keeps the ring on its own branch.

Hard point. The plant that you control is the ring with the light on. At the g26 lock its heater gain is the dark gain divided by 3 to 7, and its response is faster. A controller tuned on the dark ring has the wrong gain.

Code `selfheat.py` docstring 1–85, `solve_ring_temperature` 130–226, `implicit_step_b` 236–353; `kernel.py` `_solve_ring_eng` 627–708 and `_implicit_step_eng` 993–1130 (copies: edit both, or the parity test fails).

g26 $\eta = 0.848$, $z = 1.76$, $Z_0 = 5.704\,\text{K/mW}$, $G_{sh} = 1.76\times5.704\,\text{K/mW}\times73.89\,\text{pm/K} = 741.8\,\text{pm/mW}$, $P_{abs,peak} \approx 2.09\,\text{mW}$ per ring. At the oracle target $G_{sh}\,\partial P_{abs}/\partial\lambda_{res} = -4.7$ to -5.4, so the relief is $1+4.7 = 5.7$ to 6.4. Of the light's heat, 43 % arrives with a 0.4 μs pole.

Book $\eta = 0.3$ and one thermal resistance for heater and light ($z = 1$). So it finds only a halving of the heater gain. The stress world: `--alt uncertainty.self_heating.eta=stress_0p3`.

5.9 Bistability: two branches and a fold

When the thermal pull is strong, one heater code gives three self-consistent resonance positions (Ch10). The middle one is unstable. The outer two form a hot branch and a cold branch. A ring stays on its branch until the

branch ends at a fold. Then it jumps to the other branch. Use the normalized detuning $y = (\lambda_r - \lambda_L)/\mathrm{HWHM}$ and the pull $A = G_{sh}\, P_{abs,peak}/\mathrm{HWHM}$ (not in the textbook):

$$r_{bist} = \frac{G_{sh}\, P_{abs,peak}}{(4\sqrt{3}/9)\,\mathrm{FWHM}} > 1 \;\Rightarrow\; \text{bistable}; \qquad \text{folds: } (1+y^2)^2 + 2Ay = 0.$$

To see it, plot the self-heating shift $A/(1+y^2)$ against y, and draw the heater line $y - y_o$. One crossing gives one state. Three crossings give bistability. The direction of approach picks the branch.

Hard point. The drop peak looks like the best place to lock. In g26 the fold is only about 9 pm blue of the drop peak. A small push from the peak crosses the fold. The ring then jumps about 1.5 nm blue. The peak-seeking controllers stop at the peak, next to the fold, at about 13 dB OMA penalty.

Code `selfheat.py` `lorentzian_folds` 530–545, `bistability_ratio` 554–559; start protocols `branch_low`, `branch_high`, `laser_ramp` in `docs/ENGINE.md` section 3.

g26 $r_{bist} = 741.8 \times 2.097/(0.770 \times 340.5) = 5.93$ for ring 0, 5.93–5.98 over the bank at +4 dBm (`mrm params explain g26 bistability_ratio`).

Book A much weaker pull ($\eta = 0.3$, $z = 1$).

5.10 The heater law

A 16-bit voltage DAC drives each heater. So $P = V^2/R$, and the gain per code grows with the code. The resistance rises with the heater temperature (TCR), and that makes the gain droop at high drive. The code solves this inside the tick, with $T_H = t_{pre} + g_{self}P$. Here t_{pre} is the heater temperature without this tick's power. g_{self} (K/W) gives the rise from this tick's own power (not in the textbook):

$$u = \frac{\mathrm{code}}{2^{16}-1}, \qquad V = u\,V_{FS}, \qquad R = R_0\big[1 + \alpha_{TCR}(T_H - T_0)\big],$$

$$P = \frac{2V^2}{r_o + \sqrt{r_o^2 + 4kV^2}}, \qquad r_o = R_0\big[1 + \alpha_{TCR}(t_{pre} - T_0)\big], \qquad k = R_0\,\alpha_{TCR}\,g_{self}.$$

With $g_{self} = R_{th,H}$ and $t_{pre} = T_0$ this is the static law $P(1+yP) = V^2/R_0$, $y = \alpha_{TCR}R_{th,H}$. All heaters share one supply. Above its limit one droop factor scales every heater: $s = \min(1, \sqrt{P_{max}/\sum P}, I_{max}/\sum I)$ and $P \to s^2 P$. Here s is the droop factor, not the thermo-optic slope. The tuning efficiency is $\eta_H = \mathrm{FSR}_\lambda/P_{FSR}$.

Hard point. A heater can only heat. Code 0 is zero heat. If a disturbance heats the ring more than the heater's share of the lock temperature, no controller can cancel it. This is the limit of heater authority. For a design, linearize about the operating code with $dP/d\mathrm{code} \approx 2P/\mathrm{code}$. Schedule the gain as $1/\mathrm{code}$ (the $\sigma(u)$ of `pi_lock`).

Code `electrical.py` docstring 1–72 (actuator path), `dac_frac` 159–181, `heater_power_row` 423–531, `supply_limit_row` 534–561; `kernel.py` 2316–2383.

g26 $V_{FS} = 1.6$ V (assumed), $R_0 = 45.41\,\Omega$, $\alpha_{TCR} = 1.4 \times 10^{-3}$/K, $P_{FSR} = 25.45$ mW. So $\eta_H = 10.30\,\mathrm{nm}/25.45\,\mathrm{mW} = 0.4046$ nm/mW, and $Z_0 = 5.704$ K/mW. $y = 0.00904$/mW. Full scale: $V^2/R_0 = 56.4$ mW, and $P(1+yP) = 56.4$ gives 41 mW (about 1.6 FSR). Dark gain 0.18 pm/code at a quarter of full scale and 0.54 at three quarters (`heater_pm_per_code`). Ring 0 uses about 14 mW at the lock. DAC latency 1 μs. Supply limit 0.2 W and 0.15 A (assumed). It binds in about 1/3 of the sampled worlds.

Book TCR 10^{-3}/K on the heater's own heat only, 4.7 K/mW. Its 19 500–49 300 codes are one G26 test, not the DAC range.

5.11 Thermal dynamics and crosstalk

Heat flow obeys the same equations as an RC network. Power is the current, and temperature rise is the voltage. The model writes every path from a heat source to an observer (a point whose temperature it tracks) as a Foster sum of poles. Each pole is one RC lag. The code advances each lag exactly over a step h, for a held input (ZOH, $u_0 = u_1$) or a linear one (FOH). None of this is in the textbook:

$$Z(s) = \sum_k \frac{r_k}{1 + s\tau_k}, \qquad x_m \leftarrow a_m x_m + r_m\big(b_0 u_0 + b_1 u_1\big), \quad a = e^{-h/\tau},\; b_1 = 1 - \frac{1-a}{h/\tau},\; b_0 = \frac{1-a}{h/\tau} - a.$$

- The heater reaches its own ring through three poles: $Z_{RH}(s) = Z_0\big[\frac{f_F}{1+s\tau_F} + \frac{1-f_F-f_T}{1+s\tau_S} + \frac{f_T}{1+s\tau_T}\big]$.
- The absorbed light uses Z_{RR}. It is z times larger and has an extra fast pole.
- Ring-to-ring crosstalk is one pole per pair, $cZ_0/(1 + s\tau_{xt}(d))$. It is slower for a larger distance d.
- PDs, the die temperature sensor and the aggressor use the same distance laws.

Hard point. There is no single thermal time constant. Read the step response as a sum of exponentials. Read its Bode phase near your crossover frequency. The crosstalk in g26 is weak and slow. So it acts like a slow disturbance, not like a fast coupling between the loops.

Code `thermal.py` docstring 1–78, `foh_scalar` 164–186, `InterimRom` 761–857, `crosstalk_tau_s` 860–888, `site_fraction` 891–900, `_interim_lags` 1168–1259.

g26 Heater to own ring: 41 % at 3.18 µs, 54 % at 13.0 µs, 5 % at 3 ms. Light: $1.76Z_0 = 10.0$ K/mW, 43 % at 0.4 µs. Nearest neighbour 3 % with a 1.52 ms pole, others 1.5 %. Own PD 0.38 ms. Aggressor to ring 0: 6 % of Z_0, 0.62 ms. Die sensor at 1 mm: 1.5 %, about 9 ms. These are imec numbers, not G26 numbers. The G26 package is unknown. Corner `g26_tec_package` gives 1.0 % nearest in 0.12 ms.

Book 10 µs (75 %) + 300 µs (25 %), 10 % and 3 % crosstalk at the speed of the own heater. Corner `g26_textbook_thermal` runs that network.

5.12 The aggressor (scenario A1)

The aggressor is an on-die heater next to ring 0. Lightmatter drives it as the design-case disturbance. The model injects its power at a layout site. So ring 0, the other rings, the PDs and the die sensor each see it through their own heat path. Before the run, the code solves for the non-negative power waveform that makes ring 0 follow the published curve (not in the textbook):

$$T_{A1}(t) = 113.6 - 88.5\,\big|\sin(\pi t/P)\big|\ ^\circ\mathrm{C}, \qquad P = 130.7\ \mathrm{ms}, \qquad \max|\dot{T}| = \frac{\pi\,\Delta T}{P} = \frac{\pi \times 88.5}{0.1307} = 2127\ \mathrm{K/s},$$

$$\Delta\lambda_{tgt}(t) = s_{ref}\big(T_p(t) - T_{base}\big), \qquad s_{ref} = 15\ \mathrm{GHz/K} \cdot \frac{\lambda^2}{c} = 85.86\ \mathrm{pm/K}.$$

With the default "resonance" referencing, the printed °C become a resonance target. The code then applies the model's own $s(T)$ once, to convert that target into a ring temperature rise.

Hard point. The swing is 88.5 K × 85.86 pm/K = 7.6 nm of resonance. Ring 0's heater supplies about 81 K of its lock temperature (KNOWN_ISSUES item 38), roughly 6 nm. So at the hot end of A1 ring 0's heater reaches code 0 and stays there for about 15 ms, whatever the loop does. To see it, compare the disturbance swing in pm with the heater range in pm. The ring does not move by the full remainder: it moves red, absorbs less light and cools (section 5.8). It leaves its point by 454–473 pm.

Code `scenarios/library.py` `a1_slide10_aggressor` 133–188; `scenarios/calibrate.py` docstring 1–54, `calibrate_aggressor_power` 577–704; `experiments/examples/a1_pi_lock.yaml`.

g26 Peak rate 2127 K/s, mean rate 1354 K/s. Peak target rate about 183 pm/ms at ring 0. Site 30 µm from ring 0 and 55 µm from PD0 (assumed). Ring 0 leaves its point by 454–473 pm with every controller tried (KNOWN_ISSUES item 38). Rings 1–7 stay within about 15 pm (`pi_lock`: ring 4 14.7 pm, ring 5 6.4 pm, the others within 2.5 pm).

Book Injects the swing as a temperature at the ring only, so nothing else on the die feels it. Rate −2300 to +2200 °C/s (read from a figure).

5.13 The monitor and the ADC: what a code means

Each ring has a germanium PD on its drop port. The controller never sees the current. It sees 16-bit codes. The PD current, plus a dark current that grows with the PD temperature, is integrated over a window W. One noise sample with the exact variance is added, and the ADC converts the mean (not in the textbook):

$$i_{sig} = g_s\,\mathcal{R}\,(P_{drop} + f_{bg}P_{bus}), \qquad \bar{x} = \frac{1}{W}\sum_{k\in(s-W,s]}(i_{sig} + i_d) + \sigma\xi, \qquad \sigma^2 = \frac{S}{2T_W}, \qquad \mathrm{LSB} = \frac{I_{FS}}{2^{16}-1}.$$

- g_s is the monitor gain scale. $\mathcal{R}$ is the responsivity (A/W). f_{bg} is the background fraction (0 in g26).
- W is the window in ticks (T_W in seconds). s is the sample tick.

- $S = 2q(i_{sig} + i_d) + S_{TIA}$ is the one-sided noise density. q is the electron charge. S_{TIA} is the noise of the amplifier (TIA, transimpedance amplifier) after the PD.
- ξ is one draw from a unit normal distribution. The LSB is the current of one code.

The drop reading peaks at the resonance and is even in detuning. On one known flank the level gives the direction. On the red lock side, more heater moves the resonance further red, so the code falls ($dy/du < 0$).

Hard point. Noise is not the limit. Shot noise in one 2 μs sample is $\sqrt{2q \times 15.5\,\mu A/(2 \times 2\,\mu s)} = 1.1$ nA. The monitor slope at the lock is about 0.12 μA/pm, so the noise is about 0.01 pm (my arithmetic). Systematic errors dominate: the monitor gain (worlds scale it by 0.5–2), the dark current, and light from foreign lines. An extra current Δi moves a level lock by Δi divided by the monitor slope. So the shipped locks normalize to a peak that they measure.

Code `sensing.py` docstring 1–89, `dark_current_a` 238–248, `adc_quantize` 267–302, `monitor_tick_row` 337–467; `pi_lock.py` `block_inputs` 325–333.

g26 $\mathcal{R} = 0.95$ A/W, $I_{FS} = 40\,\mu A$, LSB $= 40\,\mu A/65\,535 = 0.610$ nA, so 1556 codes per μW of drop power. Window 2 μs, one code every 4 μs (250 kHz), latency 2 μs. Dark current 5 nA at 25 °C (×1.6 per 10 K). Drop peak about 32 μA. The monitor current at the lock misses its target (CAL-8: 18.6 against 15.5 μA).

Book No dark current, 6.4 pA/$\sqrt{\text{Hz}}$ (model 1), 15–20 μA full scale, 10 μs for window, period and delay.

5.14 Feedback for this plant

Your loop works in codes. The plant chain is: heater code → heater power ($\propto$ code^2, with TCR droop) → ring temperature (Foster poles) → resonance shift (s) → detuning (divided by the self-heating relief) → monitor code (dip slope times monitor gain). Every disturbance is slow compared with the loop, so you need integral action. `pi_lock` uses a PI controller in velocity form (it computes the change of the code, not the code). It puts the PI zero T_i on the dominant pole. It divides the error by the slope $s = dy/du$ that it measured at the set-point (here s is in ADC codes per heater code). The open loop is then about an integrator with a delay θ (not in the textbook):

$$e_k = \frac{y_k - f\,M_{pk}}{|s|}\,\sigma(u_k), \qquad \Delta u_k = k_c\Big[(e_k - e_{k-1}) + \frac{T}{T_i}e_k\Big] + v_k, \qquad k_c = 2\pi f_c T_i,$$

$$L(s) \approx \frac{\omega_c\, e^{-s\theta}}{s}, \qquad \text{PM} = 90^\circ - \frac{180^\circ}{\pi}\,\omega_c\theta, \qquad e_{ss} = \frac{\dot r}{\omega_c}\ \text{(ramp)}.$$

- f is the set-point fraction. M_{pk} is the measured peak code.
- $\sigma(u) \approx u_{ref}/u$ is the gain schedule of the voltage law. T is the controller period.
- v_k is an optional type-2 term, a second integrator (zero by default). In $L(s)$, s is the Laplace variable.
- The delay, not the thermal pole, limits the crossover. The delay is the sum of the window, the ADC latency, the wait for the next edge, the DAC latency and the hold (`timing.py`).

Hard point. The model plant has two main poles, a slow tail, a faster optical path, and a gain that changes with the code and the world. Design on the integrator-plus-delay picture. Then check it on a measured step response. An open question: a harness test holds the A1 peak rate within about 2 pm (with a 20 K swing, so the heater keeps its authority). On the engine, the A3 test (a laser sweep at 50 nm/s) leaves a 13.9 pm lag. Find the cause before you claim a bandwidth.

Code `pi_lock.py` docstring item 3 (68–86), `design_pi_gains` 366–375, `_pi_error` 423–428, `_pi_ring_step` 515–705; `core/timing.py` 1–71; `control/models.py` `linearize` 1176 (needs the `model_nominal` token).

g26 $f_c = 2.5$ kHz and $T_i = 13\,\mu s$, so $k_c = 2\pi \times 2500 \times 13\,\mu s = 0.204$ and $T/T_i = 10/13 = 0.77$. Loop delay about 12 μs, so PM $= 90^\circ - 360^\circ \times 2500 \times 12\,\mu s = 79^\circ$ before the 3.18 μs pole (my arithmetic). Heater-to-monitor gain at the lock about 10 ADC codes per heater code (a harness measured −10.0, `docs/handoff/controllers_a.md` section 4). Measure it: `pi_lock` reports `slope_per_code` in its telemetry. From the warm start `pi_lock` holds all 8 rings within 0.1 pm rms at 25 °C.

Book $K_G = 86 \times 4.7 \approx 404$ pm/mW, the same dark gain as the model. Ch13 then raises it for self-heating, $K_G/(1-k)$, with a slower pole. At the model's stable lock the gain falls 3 to 7 times and the response is faster. One pole $\tau_{th} = 10\,\mu s$. Worked design: $T_d = 30\,\mu s$, $T_s = 20\,\mu s$, $f_c = 2$ kHz.

5.15 Scoring: lock error, the oracle and the OMA penalty

The model knows the true detuning, so it scores the lock directly. The lock error is $e = d - d_{ref}$.

- A ring first locks at $|e| \le \theta_{lock}$. It unlocks at $|e| > \theta_{lock}$. It relocks at $|e| \le \theta_{lock} - h_{yst}$ (h_{yst} is the hysteresis).
- The default reference is the line ($d_{ref} = 0$). A side-of-fringe lock sits 0.3–0.5 FWHM from its line by design. So the example files use `self_reference`: d_{ref} is the median detuning over the last 5 ms of a 20 ms pre-run.
- At each instant the oracle finds the stable detuning with the best relOMA. The OMA penalty compares your relOMA with the oracle's (not in the textbook):

$$d^* = \arg \max_{d\,:\,G_{sh}\,\partial P_{abs}/\partial \lambda_{res} < 1} |T_1(d) - T_0(d)|, \qquad \text{penalty} = 10 \log_{10} \frac{\sum_t \text{relOMA}^*}{\sum_t \text{relOMA}}.$$

Hard point. The penalty is measured against the best stable point now, not against your set-point. If the conditions change and your set-point does not move, you pay a penalty with zero lock error. Compare the traces `reloma` and `oracle_reloma` of one run to see it.

Code `sim/metrics.py` docstring 1–65, `TRUTH_FIELDS` 84–109, `summarize` 740–818; `validation/oracle.py` docstring 1–75, `oma_penalty_db` 188–194; `kernel.py _oracle_a` 1534–1610; `experiment.py` 1501–1525.

g26 Threshold 5 pm, hysteresis 1 pm. Oracle refresh every 1000 ticks, 151 points over ±1.5 FWHM. Columns you will use: `max_abs_lock_error_pm`, `rms_lock_error_pm`, `unlocked_rings`, `wrong_line_rings`, `oma_penalty_db_worst`. Report the worst ring, not the mean.

Book Error over the last 20 % of the run, 5 pm rms, 19 pm peak. Penalty against the OMA at the reference point δ_{ref}, not against the best stable point now.

5.16 Uncertainty worlds

Many device numbers are unknown. The preset stores ranges or competing hypotheses for them. A world is one draw of all of them, shared by every ring of the die. Variation is separate: it spreads the geometry, heater and responsivity of each ring inside a world. A run uses the nominal world, the 13 named corners, or many sampled worlds (LHS). A claim holds only if it holds in every world b, for every ring n and every time t (not in the textbook):

$$\text{claim} \iff \max_b \max_n \max_t |e_{b,n}(t)| \le 5 \text{ pm} \quad \text{and} \quad \text{wrong_line}_b = 0 \quad \text{for all } b.$$

Hard point. No controller can hold some worlds. The heater supply limit binds in about 1/3 of the sampled worlds (KNOWN_ISSUES item 40). Ring 0 has no heater left at the hot end of A1 (code 0). Read the warm-start flags (`no_code`, `order_hopped`, `supply_limited`) in the world summary before you blame the controller.

Code `g26.yaml uncertainty` 1169–1447, `variation` 1448–1482; `experiment.py NAMED_CORNERS` 144–146; `experiments/examples/robust_pi_lock.yaml`.

g26 Worlds vary, among others: thermo-optic law (solver or H86), η (0.5–0.95), z (1–2), chip input loss (2–10 dB), Q_L (3500–4200), coupling regime, FSR, thermal family and time scales (×0.5–2), crosstalk scale, driver mode, monitor gain (0.5–2), dark current, latency scale.

Book Single nominal values with stated ranges.

6 What remains, and the first things to do

What remains to learn before you start

- ☐ Self-heating as an inner loop (sections 5.8 and 5.9). Know why the lock is on the red side and where the fold is. Know why the heater gain at the lock is the dark gain divided by 3 to 7.
- ☐ The plant chain in codes, with the light on. Measure it. From the settled warm start, apply a small open-loop heater step. Fit the gain and the poles of monitor code against heater code. Compare them with the dark gain (0.405 nm/mW) and with the 3.18 μs and 13.0 μs poles.
- ☐ The delay budget of your loop (window, ADC latency, sample age, hold, DAC latency) and the PI design with dead time. Refresh from EE: Bode plots, phase margin with dead time, the discrete integrator, the zero-order hold, lock-in (synchronous) detection.

- ☐ Start-up: warm start against cold start (`laser_ramp`, `branch_low`), bistable capture, wrong-line capture, and the line assignment of `wdm_supervisor` (acceptance test B5). Your lock law needs a hand-off: the ring on the stable branch, red of its own line.
- ☐ What your controller may know. g26 releases only the knowledge tokens `line_grid` and `ring_to_line`. Your controller must measure everything else. A grant of `heater_law`, `linewidth`, `thermal_dynamics` or `model_nominal` is a claim of factory calibration, and the report must say so.
- ☐ The lock techniques of Ch14: fraction-of-peak slope lock (`pi_lock`), dither 1f/2f (a small sine on the heater, demodulated at its frequency or twice it), bias dither, bit statistics. Do not lock on the drop peak in g26.
- ☐ How to judge a controller over the worlds: separate the worlds that the plant cannot hold from controller failures. Every result depends on thermal numbers from imec devices, not from G26.
- ☐ The traps. Run every job through `tools/memguard.py`, one at a time. The first numba compile takes about 2 minutes. After an edit to an `@njit` function outside `kernel.py`, run `tools/clear_numba_cache.py`. A scenario named in Python runs at 1.0 V bias, an experiment file at 1.05 V. Keep `g26` as the default. Change numbers per run only (`--set`, `--alt`).
- ☐ Skip for now: Ch16 (interleavers are not modelled), most of Ch08 (keep "what a picometre costs in dB"), the calibration and validation code, `mrm/waveform/`, the reinforcement-learning environment, and the bridge and HDL path (until you target an FPGA). State space, Kalman filters and MPC can wait until your PI works.

First things to do

Run these one at a time. They come from `docs/EXPLAINED.md` lines 60–78 at commit `3db791e`, where they were tested. The current `EXPLAINED.md` has only two of them: the first run (section "Try it") and the first `my_lock` run (section "Use your own controller"). Copy the commands from `docs/guide/mrm_guide.tex`: a line break in this PDF adds a ↪ sign.

```
cd ~/mrm-model

# a shipped controller (pi_lock_c) on g26, 40 ms; it prints a report and writes runs/<UTC time>_<hash>/
.venv/bin/python tools/memguard.py -- .venv/bin/mrm run experiments/examples/steady_pi_lock.yaml

# copy the template, then edit step() in my_lock.py
cp examples/controllers/template_python_controller.py examples/controllers/my_lock.py

# your controller: steady, then under the A1 aggressor (with a controller parameter changed)
.venv/bin/python tools/memguard.py -- .venv/bin/python -m mrm.cli run experiments/examples/steady_pi_lock.yaml --set
    ↪ controller.name=examples.controllers.my_lock:MyIntegralLock
.venv/bin/python tools/memguard.py -- .venv/bin/python -m mrm.cli run experiments/examples/a1_pi_lock.yaml --set controller.
    ↪ name=examples.controllers.my_lock:MyIntegralLock --set controller.params.k_i_codes=800

# your controller over the uncertainty: nominal world, named corners, 128 sampled worlds (about 3 minutes)
.venv/bin/python tools/memguard.py -- .venv/bin/python -m mrm.cli run experiments/examples/robust_pi_lock.yaml --set
    ↪ controller.name=examples.controllers.my_lock:MyIntegralLock
```

- In `my_lock.py`, rename the class and the name in `@register(...)`. Use the new class name after the colon.
- The plot script was in `docs/EXPLAINED.md` lines 96–119 at commit `3db791e`. The current `EXPLAINED.md` does not have it. To read the old file, run `git show 3db791e:docs/EXPLAINED.md`.
- Under A1, ring 0 leaves its point by 445 to 475 pm (the template gives 445 pm). That is the limit of heater authority, not of your loop.
- The robust run takes about 2.5 minutes. It prints FAIL and exits with code 1 when one world breaks a criterion. That is a result, not a crash.
- The viewer: `cd webviz/site && python3 -m http.server 8000`, then open `http://localhost:8000`.